

EntropyX: A Hybrid C++ Cryptographic Password Suite with Shannon Entropy Analysis and a Zero-Knowledge Client-Side Vault

Nigam Mehta

mehtanigam3024@gmail.com

Pursuing B.Tech in Electronics Engineering (VLSI Design and Technology)

Abstract—EntropyX is a hybrid cryptographic password-generation and security suite combining a native C++14 microservice with a React 19 and Tailwind CSS web interface. It provides cryptographically secure password and passphrase generation, unbiased rejection sampling, SHA-256 based global deduplication, Bloom-filter acceleration, Shannon entropy estimation, crack-time analysis, predictable-pattern detection, breach auditing, and a client-side zero-knowledge password vault. Local execution uses Windows CryptoAPI randomness while cloud deployment can fall back to the browser Web Crypto API. The vault derives a 256-bit AES-GCM key from a master password using PBKDF2-HMAC-SHA256 with 100,000 iterations and a 128-bit salt, and stores authenticated ciphertext in browser localStorage. The project also contains a multithreaded benchmark and a 32-bit Galois LFSR reference model.

Index Terms—CSPRNG, password generation, Shannon entropy, AES-256-GCM, PBKDF2, zero-knowledge vault, Bloom filter, SHA-256, React, C++14.

I. INTRODUCTION

Weak and reused passwords remain a major security problem. A useful password-security application must provide strong randomness, measurable strength, predictable-pattern detection, uniqueness tracking, and secure local storage. EntropyX addresses these requirements with a hybrid native C++14 engine and a modern React frontend. The repository describes the system as a high-performance cryptographic suite with a client-side encrypted vault and hybrid local/cloud generation architecture. [?]

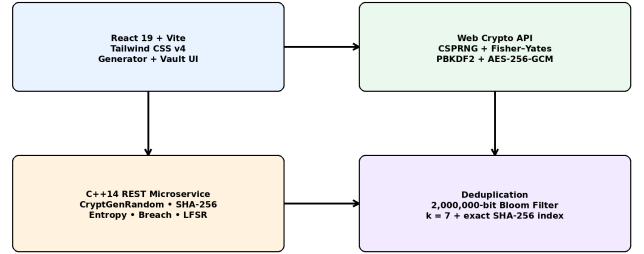
II. OBJECTIVES

The project aims to (1) generate secure passwords and passphrases, (2) eliminate modulo bias, (3) calculate entropy and crack time, (4) detect keyboard walks, sequences and repetitions, (5) provide persistent deduplication, (6) implement a zero-knowledge vault, (7) expose the native engine through REST, and (8) support local and Vercel deployment.

III. ARCHITECTURE

The frontend uses React 19, Vite and Tailwind CSS v4. The backend is a C++14 HTTP service, normally on port 8080, containing password generation, entropy analysis, SHA-256, Bloom filtering, deduplication, breach auditing and an LFSR model. The frontend first attempts the native API and falls back to Web Crypto when unavailable: UI $\rightarrow$ {C++ service or Web Crypto}.

EntropyX — Hybrid Security Architecture



Local mode uses the native service; cloud mode falls back to browser Web Crypto.

Fig. 1. EntropyX hybrid architecture derived from the implemented repository.

TABLE I
TECHNOLOGY STACK

Layer	Technology
Frontend	React 19, Vite 6, Tailwind CSS v4
Backend	C++14 native REST service
Local CSPRNG	Windows CryptoAPI CryptGenRandom
Cloud CSPRNG	W3C Web Crypto API
Hashing	SHA-256
Deduplication	2,000,000-bit Bloom filter + exact hash set
Vault KDF	PBKDF2-HMAC-SHA256, 100,000 iterations
Vault cipher	AES-256-GCM, 96-bit IV
Deployment	Vercel-compatible frontend

IV. CSPRNG PASSWORD GENERATION

The native engine obtains 32-bit random values using Windows CryptoAPI CryptGenRandom and falls back to `std::random_device` if the provider is unavailable. The browser uses `crypto.getRandomValues()`. Uppercase, lowercase, numeric and symbol pools are configurable and ambiguous characters can be removed. At least one character from each active class is inserted before the remaining positions are filled from the combined pool. A cryptographic Fisher–Yates shuffle then randomizes the final sequence.

A. Unbiased Rejection Sampling

For a character pool of size M , direct modulo mapping can introduce bias. EntropyX computes $L = 2^{32} - (2^{32} \bmod M)$ and accepts a random 32-bit value X only when $X < L$.

The selected index is then $X \bmod M$, producing an unbiased mapping over the requested pool.

V. PASSPHRASE GENERATION

The native engine also supports passphrases using a curated wordlist. Words are selected with the CSPRNG, optionally capitalized, joined with a configurable separator, and optionally followed by a three-digit number. The implementation estimates passphrase entropy as $E \approx 12.9w + 9.9I_N$, where w is the word count and I_N indicates inclusion of the number.

VI. ENTROPY AND CRACK-TIME ANALYSIS

The project uses the pool-size model $E = L \log_2(N)$, where L is length and N is the effective character pool. When no pool is supplied, the analyzer estimates $N = 26I_U + 26I_L + 10I_D + 32I_S$. Strength is classified as Weak below 40 bits, Fair from 40–64, Strong from 65–79, and Very Strong at 80 bits or above. Crack time assumes 10^{10} offline guesses/s and an average search of half the space: $T = 2^E / (2 \times 10^{10})$ seconds. This is an educational estimate, not a universal prediction.

VII. PATTERN DETECTION

The analyzer detects common keyboard walks such as qwerty and asdfgh, ascending or descending runs of four or more characters, and three or more repeated consecutive characters. Warnings are accompanied by suggestions such as increasing length and avoiding predictable patterns.

VIII. PERSISTENT DEDUPLICATION

Each generated password is hashed with SHA-256. A 2,000,000-bit Bloom filter with $k = 7$ hash functions provides a fast first check, followed by an exact hash-set lookup whenever the Bloom filter reports a possible match. A candidate is registered only after the exact check succeeds. The approximate Bloom false-positive probability is $p \approx (1 - e^{-kn/m})^k$. Persistent 64-character hexadecimal SHA-256 digests are loaded at startup and appended for newly registered passwords. The Bloom filter therefore accelerates lookup without turning false positives into false uniqueness claims.

IX. ZERO-KNOWLEDGE VAULT

Vault contents are encrypted in the browser using the W3C Web Crypto API. A random 128-bit salt is used with PBKDF2-HMAC-SHA256 for 100,000 iterations to derive a 256-bit AES-GCM key: $K = \text{PBKDF2}(P, S, 100000, 256)$. A random 96-bit IV is used for AES-256-GCM authenticated encryption. The ciphertext, salt, IV, KDF parameters, item count and timestamp are stored in localStorage. The React application retains decrypted items and the master password in memory and immediately clears them when switching away from the vault. There is intentionally no master-password recovery backdoor.

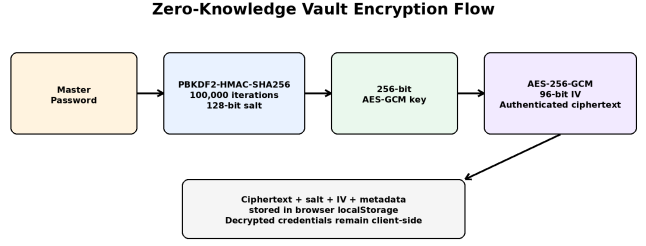


Fig. 2. Client-side zero-knowledge vault encryption flow.

TABLE II
NATIVE REST ENDPOINTS

Endpoint	Purpose
/api/health	Health and deduplication statistics
/api/generate	Unique password generation
/api/generate-passphrase	Unique passphrase generation
/api/analyze	Entropy and pattern analysis
/api/benchmark	Multithreaded throughput benchmark
/api/check-breach	Breach audit / prefix mode
/api/vlsi-model	32-bit Galois LFSR trace

X. REST API

The C++ service implements GET /api/health, POST /api/generate, POST /api/generate-passphrase, POST /api/analyze, POST /api/benchmark, POST /api/check-breach and GET /api/vlsi-model. Generation responses include the password, SHA-256 digest, entropy, strength, crack time, pool size, uniqueness status, attempts and latency. The frontend service applies a short timeout and switches to Web Crypto if the native service is unreachable.

XI. BREACH AUDITING

The backend contains an offline breach-signature auditor and supports a prefix-based k-anonymity style query. A client can provide a short hash prefix and receive matching suffix/frequency information, reducing the need to transmit the complete plaintext password to the service.

XII. MULTITHREADED BENCHMARK

The benchmark creates asynchronous workers with std::async. Each worker creates an independent PasswordEngine and generates 16-character passwords. For N generated passwords in t seconds, throughput is $R = N/t$. Requests are capped at 200,000 generations.

XIII. VLSI REFERENCE MODEL

The /api/vlsi-model endpoint exposes a 32-bit Galois LFSR reference model with feedback mask 0x80200003. The modeled maximal period is $2^{32} - 1$ cycles. The endpoint returns cycle number, hexadecimal state, binary state and output bit, connecting the software security project with hardware-oriented pseudorandom sequence concepts.

XIV. DEPLOYMENT

The repository provides a Windows run.bat launcher that can compile the native backend, start the REST service, launch Vite and open the browser. Manual execution uses the C++ server followed by npm install and npm run dev in the frontend. The Vercel configuration allows cloud deployment where Web Crypto provides the generator fallback.

A. Live Website

Figure 3 shows the deployed EntropyX password-generator interface, including character-class controls, password length, strength feedback, and access to the vault. The live application is available online [?].

Live application: <https://entropyx-password-suite.vercel.app/>
Author portfolio: <https://nigam-portfolio.vercel.app/>

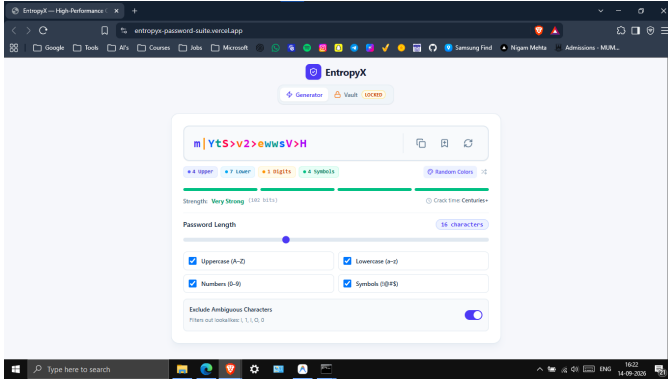


Fig. 3. Screenshot of the live EntropyX website showing the password generator, character-class settings, entropy-based strength indicator, and vault navigation.

XV. SECURITY ANALYSIS

Important security properties include CSPRNG generation, unbiased selection, cryptographic shuffling, SHA-256 deduplication, authenticated AES-GCM storage, PBKDF2 key derivation, volatile decrypted state, auto-locking, and absence of a recovery backdoor. These properties do not by themselves make the application a certified production password manager. Browser compromise, XSS, malicious extensions, localStorage exposure, device compromise, dependency attacks, and threat-model assumptions remain relevant.

XVI. TESTING AND RESULTS

Representative tests include health checks, generation with each character-class combination, ambiguous-character exclusion, modulo-bias validation, duplicate re-rolls, entropy thresholds, keyboard-pattern detection, vault encryption/decryption, wrong-password failure, ciphertext tampering failure, auto-lock, cloud fallback, benchmark throughput and breach-prefix queries. The repository contains the corresponding implementation paths. Further quantitative evaluation should measure latency, Bloom false-positive rate at different occupancies, multithreaded throughput, and vault cryptographic latency on target hardware.

TABLE III
REPRESENTATIVE VERIFICATION TESTS

Test	Expected result
CSPRNG generation	Non-empty cryptographic password
Character controls	Requested active classes represented
Deduplication	Previously registered digest re-rolled
Entropy analysis	Strength, warnings and crack time returned
Vault encryption	Authenticated ciphertext stored
Wrong master password	Decryption rejected
Tampered ciphertext	Authentication failure
Auto-lock	Decrypted state cleared
Cloud fallback	Web Crypto generation works
Benchmark	Throughput and worker count returned

XVII. LIMITATIONS

The native CSPRNG path is Windows-specific. Cloud fallback does not provide the native persistent global deduplication service. The entropy model is simplified and crack-time estimates depend on attacker assumptions. Browser localStorage is not hardware-backed secure storage. The custom HTTP/JSON implementation would need extensive security review for production. The project also intentionally has no master-password recovery mechanism.

XVIII. FUTURE SCOPE

Future work can include cross-platform OS randomness, stronger memory-hard KDF options, hardware-backed credentials, CSP and Trusted Types hardening, cryptographic property testing, distributed deduplication, authenticated service-to-service communication, richer breach analytics, and reproducible performance benchmarks.

XIX. CONCLUSION

EntropyX demonstrates an integrated security application combining native C++ cryptographic generation, deterministic password analysis, persistent deduplication, breach auditing, multithreaded benchmarking, a modern React interface, and a client-side AES-256-GCM vault. Its hybrid deployment model allows local native operation while preserving useful cloud functionality through browser Web Crypto. The project provides a strong academic demonstration of cryptographic programming, web engineering, algorithms, secure storage, concurrency and deployment, while its documented limitations clearly distinguish it from a production-certified password-management system.

REFERENCES

- [1] NIST, "Digital Identity Guidelines: Authentication and Lifecycle Management," SP 800-63B.
- [2] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D.
- [3] W3C, "Web Cryptography API," <https://www.w3.org/TR/WebCryptoAPI/>
- [4] Meta Open Source, "React Documentation," <https://react.dev/>